

CSCI 240 -- PA 4

Stacks and Queues

Feel free to discuss and help each other out but does not imply that you can give away your code or your answers! Make sure to read all instructions before attempting this lab. You cannot work with a lab partner for this assignment.

You must use an appropriate provided template from Canvas and output "Author: Your Name(s)" for all your programs. If you are modifying an existing program, use "Modified by: Your Name(s)".

Exercise 1: You must use either existing C++ language **stack** class or Java language **Stack** class to solve the "Balancing Symbols" problem. The symbols are (), [], < >, and { }, and each opening symbol must have a corresponding closing symbol as well as in correct order. Ignore operands and arithmetic operators since they are not relevant to our problem. **You can assume each token is separated by spaces so you can input each item as one string.** For each invalid situation, output a reasonable explanation. Try the 4 examples below:

- { (a + b) * c1 } – valid
- { (a + b) * c1] – invalid, closing symbol does not match opening symbol
- (< a + b > * c1) / 15) – invalid, no matching opening symbol
- (((a + b) * c1) / 15) – valid
- (< 12 % 3 > + 4 -- – invalid, no matching closing symbol

Exercise 2: You must define and implement your own **MyArrayStack** class. The MyArrayStack class supports standard basic stack operations below, and you must implement it with an array with up to 5 elements. You must create a class template in C++ or generic class in Java. Refer to the book's implementation if you need help with the code.

Interface for class MyArrayStack:

- size() – return the number of elements
- empty() – return true if stack is empty
- top() – return the top element, throw an exception if it is empty
- push(e) – push element onto the stack, throw an exception if it is full
- pop() – remove and return top element, throw an exception if it is empty

Set up a function or static method that receives a string representing a valid postfix expression and it returns an integer result. Your function/method uses MyArrayStack to create an integer stack object so it can be used to evaluate a postfix expression (see an

operand (17 or -3) -- push; see an operator (+, -, *, /) – pop twice, evaluate, then push result; result is the one top element when we are done with postfix expression). **You can assume each token is separated by spaces so you can extract each item as one string first and convert to an int if applicable.**

Test your function/method and MyArrayStack class with the following postfix expressions by using a test driver:

```
17 2 3 + / 13 -  
5 2 3 + *  
2 3 2 / *  
-23 123 + 2 *  
17 2 3 4 5 * 6 7 * + / + * -
```

Answers for the postfix expressions above:

```
-10  
25  
2  
200  
stack overflow
```

Exercise 3: You must define and implement your own **MyLinkedList** class. The MyLinkedList class supports standard basic queue operations below, and you must implement it with a linked list. You must create a class template in C++ or generic class in Java. Refer to the book's implementation if you need help with the code.

Interface for class MyLinkedList:

- size() – return the number of elements
- empty() – return true if queue is empty
- front() – return the first element, client should only use it when queue is not empty
- enqueue(e) – add element to the rear of the queue
- dequeue() – remove and return the first element, client should only use it when queue is not empty

You need to write a program that simulates a simple queue by the operating system. The OS processes the jobs in a round robin fashion until all the jobs are done. See below for an algorithm.

```
Generate 5 jobs with a random processing unit between 1 and 6  
Add all jobs (jobNum 1 to 5) to the queue  
While queue is not empty  
    Dequeue 1 item and serve it for 1 unit  
    If remaining unit > 0 then  
        Enqueue it back  
    Else  
        Display "Done with job", jobNum
```

```
End if
End while
```

Make sure to provide a reasonable output. Use 1, 2, 3, ..., to label each processing cycle and a job number of 1, 2, 3, and so forth. **Try both MyLinkedList (using the provided 5 values) and the queue from the C++ or Java language (using random numbers) so there are 2 sets of output.** Think about the data that each queue entry should hold.

Sample output when hardcoding the 5 jobs with 2, 1, 2, 1, 1 units for testing:

Using MyLinkedList with 2, 1, 2, 1, 1.

```
Enqueue job 1 with 2 units
Enqueue job 2 with 1 units
Enqueue job 3 with 2 units
Enqueue job 4 with 1 units
Enqueue job 5 with 1 units
```

```
1 Processing job 1
2 Processing job 2
  Done with job 2
3 Processing job 3
4 Processing job 4
  Done with job 4
5 Processing job 5
  Done with job 5
6 Processing job 1
  Done with job 1
7 Processing job 3
  Done with job 3
Done with all jobs
```

Question 1: Given a choice between an array and a linked list, which one is better to implement a queue? Explain.

Question 2: Would you consider an array implementation for the deque? Explain why or why not.

You can earn EC for only one of the options below:

Extra Credit Option 1 (3 points): Set up a function or static method that receives a string representing an infix expression and it returns an equivalent postfix expression. You only need to deal with the following operators: +, -, *, /. Your function/method uses a stack of characters to convert an infix expression to a postfix expression. Use MyArrayStack class from exercise 2.

- Test your function/method with the following infix expressions by using the same test driver from exercise 2:

```

17 / ( 2 + 3 ) - 13
5 * ( 2 + 3 )
2 * ( 3 / 2 )
( -23 + 123 ) * 2
17 - 2 * 3 / ( 4 * 5 + 6 * 7 )

```

- Answers for the infix expressions above:

```

17 2 3 + / 13 -
5 2 3 + *
2 3 2 / *
-23 123 + 2 *
17 2 3 4 5 * 6 7 * + / + * -

```

Extra Credit Option 2 (2 points): Provide a linked implementation of a deque and name it **MyLinkedDeque** (can use your own **doubly** linked list or the one from the book). It can either be a template/generic class or a class that works with strings. Use a test driver to try out your MyLinkedDeque by adding and removing values from both ends. Try the following test cases: insert front (“John”), insert front(“Jane”), insert rear(“Jo”), remove rear, remove front, output size, and output front item.

Fill out and turn in the PA submission file for this assignment (save as PDF format).